

# SIMATS ENGINEERING

## CO1 - ASSESSMENT TOOL 2

### ERROR ANALYSIS TASK

|                            |                              |
|----------------------------|------------------------------|
| <b>Name of the Student</b> | MADDIRALA BALA MURALIKRISHNA |
| <b>Reg. No</b>             | 192325015                    |

**COURSE NAME: Deep Learning**

**COURSE CODE: MLA0403**

**FACULTY : Dr . Arul Raj A.M**

#### COURSE OUTCOME COVERED

**CO 1:** Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini-Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

**CO1 Weightage: 15%**

**Assessment Tool Weightage: 30%**

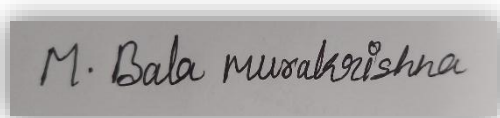
**Duration: 30 minutes**

**Marks: 25**

#### Code of Conduct:

I Maddirala Bala Muralikrishna (192325015) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.



MADDIRALA BALA MURALI KRISHNA

**Signature of the student:**

**Name of the student:**

## QUESTIONS: 05    Total Marks: 25

### 1. Error Analysis in Maximum Likelihood Estimation (MLE)

*A student builds a binary classifier using Maximum Likelihood Estimation. The likelihood function is incorrectly written as the sum of probabilities instead of the product. The model produces poor parameter estimates even with large data.*

**a) Identify the conceptual error in the likelihood formulation.**

**Answer:**

- For independent and identically distributed (i.i.d.) observations  $x_1, x_2, \dots, x_n$ , the joint probability of observing all of them together is the **product** of their individual probabilities,  $P(x_1, \dots, x_n; \theta) = \prod_i p(x_i; \theta)$ , because independence means  $P(A \cap B) = P(A) \cdot P(B)$ , not  $P(A) + P(B)$ .
- Writing the likelihood as a sum,  $L(\theta) = \sum_i p(x_i; \theta)$ , instead treats the observations as mutually exclusive alternatives (as in  $P(A \cup B)$  for disjoint events), which is not what a dataset of jointly observed samples represents.
- This is a fundamental confusion between the addition rule (used for the union of mutually exclusive events) and the multiplication rule (used for the joint occurrence of independent events).

**b) Explain how this mistake affects parameter estimation.**

**Answer:**

- A sum of probabilities is dominated by whichever single term  $p(x_i; \theta)$  is largest, since increasing one term increases the sum almost as much regardless of the others; the optimizer effectively fits  $\theta$  to maximize the fit of only the easiest point rather than balancing the fit across the entire dataset.
- The true joint likelihood shrinks multiplicatively toward zero as  $n$  grows (since each  $p(x_i; \theta) \leq 1$ ), which correctly reflects that observing many specific data points jointly is a rare, highly specific event. A sum does not shrink this way, so it does not correctly penalize parameter values that fit only part of the data well.

- As a result, the gradient of the (incorrect) objective does not properly aggregate information from all  $n$  samples, so the estimator is biased and inconsistent — more data does not reliably drive  $\hat{\theta}$  toward the true parameter, which explains why performance stays poor even with large datasets.

**c) Suggest the correct mathematical formulation and reasoning.**

**Answer:**

For i.i.d. samples  $x_1, \dots, x_n$  with parameter  $\theta$ , the likelihood is the product of individual probabilities (or densities):

$$L(\theta) = \prod_{i=1}^n p(x_i; \theta)$$

- This follows directly from the definition of statistical independence: the joint pmf/pdf of independent random variables factorizes into the product of their marginal pmfs/pdfs.
- MLE then chooses  $\hat{\theta} = \operatorname{argmax}_{\theta} L(\theta)$ , i.e., the parameter value under which the observed sample, taken as a whole, was most probable — this is only meaningful when the joint probability (product) is used, not the sum.

**d) How would using log-likelihood fix computational issues?**

**Answer:**

- Taking logs converts the product into a sum:  $\log L(\theta) = \sum_i \log p(x_i; \theta)$ , which is analytically and computationally easier to differentiate — the derivative becomes a simple sum of per-sample derivatives instead of requiring the product rule across  $n$  terms.
- Since the log function is strictly monotonically increasing,  $\operatorname{argmax}_{\theta} \log L(\theta) = \operatorname{argmax}_{\theta} L(\theta)$ , so the maximizer is unchanged — log-likelihood is a safe reformulation, not an approximation.
- Numerically, multiplying many probabilities (each  $< 1$ ) causes the product to underflow to 0 in floating-point arithmetic for even moderately large  $n$ ; summing their logs avoids this underflow and keeps the objective in a numerically stable range.
- Log-likelihood also decomposes cleanly for exponential-family distributions (e.g., the Gaussian log-likelihood), turning exponentials into linear/quadratic terms that are simple to optimize with
- gradient-based methods.

## 2. Error Diagnosis in Perceptron Learning

*While implementing a Perceptron, a student observes that the model fails to converge even after many iterations on a linearly separable dataset.*

**a) List possible implementation errors in the learning algorithm.**

**Answer:**

- Incorrect weight-update rule — e.g., using  $w \leftarrow w - \eta \cdot y \cdot x$  instead of the correct  $w \leftarrow w + \eta \cdot y \cdot x$  for a misclassified point, which reverses the direction of learning.
- Updating the weights on every sample (including correctly classified ones) instead of only on misclassified points, which perturbs an already-correct boundary.
- Missing or incorrectly updated bias term, so the decision boundary is forced through the origin and cannot separate data that requires an offset.
- Inconsistent label encoding (e.g., mixing  $\{0,1\}$  and  $\{-1,+1\}$  conventions) so the sign-based update rule computes the wrong update direction for some samples.
- No proper convergence check (e.g., stopping only after a fixed epoch count rather than checking for zero misclassifications), which can make a perceptron that has actually converged look like it hasn't.

**b) Analyze how incorrect learning rate or weight updates affect convergence.**

**Answer:**

- The Perceptron Convergence Theorem guarantees that for linearly separable data, the standard update rule converges in a finite number of steps regardless of the (positive) learning rate value — so persistent non-convergence on genuinely separable data points to an error in the update logic, not merely a poorly tuned rate.
- A sign error in the update (adding instead of subtracting, or vice-versa) makes the weight vector move away from the correct separating direction on every misclassified point, so the algorithm can cycle indefinitely without ever converging.
- An excessively large learning rate can cause the weight vector to overshoot past a good separating boundary each time a point is misclassified, producing oscillation between different misclassified points rather than settling; an excessively small rate makes each correction negligible, so convergence, while theoretically guaranteed, may take far more iterations than the training budget allows and appear as non-convergence.

**c) Identify dataset-related issues that may cause this problem.**

**Answer:**

- Mislabelled samples or duplicate feature vectors with conflicting labels effectively make the 'linearly separable' dataset non-separable in practice, which would cause the perceptron to oscillate forever.
- Unnormalized or vastly differently-scaled features can cause the weight update to be dominated by high-magnitude features, slowing effective convergence even though the theoretical guarantee still holds.
- A fixed, non-shuffled presentation order can create cyclic update patterns on certain data arrangements, making convergence appear stalled within a limited number of epochs even though it is progressing slowly.
- Near-zero margin between classes (points very close to the true boundary) means many epochs may be needed before the weight vector rotates enough to classify every point correctly, since the number of updates required by the convergence bound grows as the margin shrinks.

**d) Propose modifications to ensure convergence.**

**Answer:**

- Fix and verify the update rule against the standard form: if  $y_i(w \cdot x_i + b) \leq 0$ , set  $w \leftarrow w + \eta \cdot y_i \cdot x_i$  and  $b \leftarrow b + \eta \cdot y_i$ ; leave weights unchanged for correctly classified points.
- Explicitly check for zero misclassifications on a full pass as the stopping criterion, rather than relying on a fixed epoch limit.
- Normalize/standardize input features so that no single feature dominates the update magnitude.
- Shuffle the training data at the start of every epoch to avoid pathological cyclic update sequences.
- Verify the dataset is genuinely linearly separable (e.g., by fitting an SVM or checking with a convex-hull test) and remove or relabel any conflicting points.
- If near-separability is a concern, use the Pocket Algorithm, which keeps the best-performing weight vector seen so far and is more robust when the margin is very small or the data is only approximately separable.

### 3. Backpropagation Gradient Error Analysis

*A Multilayer Perceptron trained using Backpropagation shows increasing loss during training instead of decreasing.*

**a) Identify possible mathematical or coding errors in gradient computation.**

**Answer:**

- A sign error in the parameter update — using  $w \leftarrow w + \eta \nabla L$  instead of the correct  $w \leftarrow w - \eta \nabla L$  — causes every step to move **up** the loss surface instead of down, which alone is sufficient to make loss increase monotonically.
- A learning rate that is far too large overshoots the minimum at every step; on a curved loss surface this can cause the loss to grow rather than shrink even with a mathematically correct gradient direction.
- Incorrect application of the chain rule during backpropagation — missing a derivative term, using the wrong matrix orientation (forgetting to transpose weight matrices when propagating error backward), or reusing pre-activation values incorrectly — yields gradients that do not point in a descent direction.
- Mismatched loss-function derivative (e.g., using the wrong derivative for cross-entropy versus mean-squared error) produces a gradient that does not correspond to the actual loss being reported, so ‘descending’ the computed gradient does not reduce the reported loss.
- Poor weight initialization (e.g., very large random weights) combined with unbounded activations can cause activations and gradients to explode from the very first forward/backward pass.

**b) Explain the role of activation functions in gradient flow.**

**Answer:**

- Backpropagation computes each layer's gradient via the chain rule, multiplying the local derivative of the activation function at each layer with the gradient flowing back from the layer above — the activation's derivative therefore acts as a multiplicative gate on the gradient signal at every layer.
- Sigmoid and tanh saturate for large  $|\text{input}|$  (their derivative approaches 0 in those regions), so stacking several such layers multiplies many small numbers together, shrinking the gradient exponentially with depth (vanishing gradients) and effectively freezing early-layer learning.

- ReLU has a derivative of exactly 1 for positive inputs, which avoids the multiplicative shrinkage that sigmoid/tanh cause, but its derivative is 0 for negative inputs, which can silence ('kill') neurons whose input is consistently negative.
- The choice of activation therefore directly determines whether gradient magnitude is preserved, shrunk, or amplified as it is propagated backward through the network's depth.

**c) *Analyze how vanishing or exploding gradients affect training.***

**Answer:**

- Vanishing gradients: repeated multiplication of derivatives with magnitude  $< 1$  across many layers makes the gradient reaching early layers exponentially small, so those layers' weights barely update; training appears to stall or converge extremely slowly, and the loss curve can look flat or noisy rather than steadily decreasing.
- Exploding gradients: repeated multiplication of derivatives/weights with magnitude  $> 1$  causes the gradient magnitude to grow exponentially with depth, producing very large parameter updates at each step; this directly explains an increasing loss, since the weights are pushed far past any reasonable minimum and the loss surface is essentially overshoot every iteration, sometimes leading to numerical overflow (NaN losses).
- Both effects are compounded by network depth: the more layers the chain rule must multiply through, the more extreme either the shrinkage or amplification becomes.

**d) *Suggest techniques to fix the issue (e.g., normalization, initialization).***

**Answer:**

- Use principled weight initialization (Xavier/Glorot for sigmoid/tanh, He initialization for ReLU) so that activation and gradient variance is preserved across layers at the start of training.
- Apply gradient clipping to cap the gradient norm, directly preventing exploding updates regardless of their theoretical magnitude.
- Use Batch Normalization (or Layer Normalization) to keep layer inputs in a well-scaled range throughout training, which stabilizes both forward activations and backward gradients.
- Prefer non-saturating activations such as ReLU or its variants (Leaky ReLU, ELU) over sigmoid/tanh in deep hidden layers to reduce vanishing-gradient risk.
- Reduce the learning rate (or use a warm-up/decay schedule) if the loss is diverging due to overshooting.

- Verify the backward-pass implementation with numerical gradient checking to rule out coding errors before tuning hyperparameters.

## 4. Gradient Descent vs SGD Error Investigation

*A model trained using Gradient Descent converges very slowly, while the same model with Stochastic Gradient Descent shows unstable loss fluctuations.*

**a) Explain why batch gradient descent is slow in large datasets.**

**Answer:**

- Batch (full) gradient descent computes the gradient by summing/averaging the loss derivative over every training example before taking a single update step, so each iteration costs  $O(n)$  in the dataset size  $n$ ; for large  $n$  this makes even one parameter update expensive.
- Because only one update happens per full pass over the data, the number of updates performed per unit of computation time is very low compared to methods that update more frequently, so wall-clock convergence is slow even though the update direction is the most accurate available.
- Very large datasets may not fit in memory at once, requiring costly disk I/O or batching just to compute a single exact gradient, further slowing each iteration.

**b) Analyze the cause of fluctuations in SGD.**

**Answer:**

- SGD estimates the true gradient using only a single randomly chosen sample at each step; this single-sample estimate has high variance relative to the true (full-dataset) gradient, since one point's loss surface is generally a poor proxy for the whole dataset's loss surface.
- Consecutive updates are therefore computed from different, noisy gradient estimates that can point in substantially different directions, so the parameters do not move smoothly toward the minimum but instead jitter around it — the loss curve fluctuates step to step even as the general trend is downward.
- Near the minimum, this per-sample noise does not vanish (it depends on the variance of individual sample gradients, not on proximity to the optimum), so SGD tends to fluctuate around the minimum rather than settling exactly at it unless the learning rate is decayed.



**c) Compare the error behavior of Mini-Batch Gradient Descent.**

**Answer:**

- Mini-batch gradient descent averages the gradient over a batch of  $b$  samples ( $1 \ll b \ll n$ ); by properties of averaging independent noisy estimates, the variance of the gradient estimate shrinks roughly by a factor of  $1/b$  compared to single-sample SGD, giving a smoother, less erratic loss trajectory than pure SGD.
- Unlike full-batch GD, mini-batch still performs many updates per pass over the dataset ( $n/b$  updates per epoch), so it retains much of SGD's faster wall-clock progress and ability to escape shallow local structure, while being noticeably more stable than single-sample updates.
- Its error behavior is therefore intermediate: noisier than full-batch GD but much smoother than SGD, and it benefits additionally from vectorized/parallel hardware computation on the batch, making each update more computationally efficient than either extreme.

**d) Recommend the best approach and justify with error trade-offs.**

**Answer:**

- Mini-batch gradient descent (typically with batch sizes in the tens-to-low-hundreds, combined with an adaptive optimizer such as Adam or RMSProp and/or a learning-rate decay schedule) is the recommended approach for most practical training scenarios.
- It resolves the slowness of full-batch GD by updating far more frequently per epoch, while resolving the instability of pure SGD by averaging out much of the single-sample gradient noise.
- The residual noise retained from using a finite batch (rather than the full dataset) is not purely a drawback — moderate gradient noise can help the optimizer escape sharp, poorly-generalizing minima and land in flatter regions of the loss surface associated with better generalization, so the trade-off in mini-batch size is between per-step computational efficiency, gradient-estimate variance, and generalization quality, not just raw convergence speed.

## 5. Curse of Dimensionality Error Analysis

*A machine learning model built using high-dimensional data performs well on training data but poorly on test data due to overfitting, related to the Curse of Dimensionality.*

**a) Explain how high dimensionality increases model error.**

**Answer:**

- As the number of features  $p$  grows, the volume of the feature space grows exponentially with  $p$ , so a fixed number of training samples  $n$  covers an ever-shrinking fraction of that space — the data becomes extremely sparse relative to the space the model must generalize over.
- With  $p$  large relative to  $n$  (as in a high-dimensional setting), a sufficiently flexible model has enough free parameters/degrees of freedom to fit spurious combinations of features that happen to separate or predict the training labels exactly, without those combinations reflecting any real underlying relationship.
- Because the training data covers the space so sparsely, the model's fitted function is only well-constrained near the observed training points; in the vast unobserved regions of the high-dimensional space (including where most test points fall) its predictions are essentially unconstrained extrapolations, producing large test error even though training error is low.

**b) Identify signs of overfitting in this scenario.**

**Answer:**

- A large and persistent gap between training performance (near-perfect accuracy / very low loss) and test performance (much lower accuracy / much higher loss).
- Training loss continues to decrease over epochs while validation/test loss decreases initially and then rises — the classic overfitting curve.
- High sensitivity of the learned model to small perturbations of the training set (e.g., retraining on a slightly different subsample yields a very different decision boundary), indicating the model has memorized idiosyncrasies of the specific sample rather than learning stable, generalizable structure.
- An unusually complex decision boundary or a very large number of 'important' features/weights relative to the amount of data available.

**c) *Analyze why distance-based learning fails in high dimensions.***

**Answer:**

- Distance-based methods (k-NN, k-means, kernel methods using Euclidean/RBF distance) rely on the assumption that ‘nearby’ points are meaningfully more similar than ‘far’ points.
- In high dimensions, pairwise distances between randomly distributed points concentrate: as  $p$  grows, the ratio (max distance – min distance) / min distance across a dataset tends toward zero, meaning almost all points become nearly equidistant from any given query point.
- When every point is roughly the same distance away, the notion of a meaningful ‘nearest neighbor’ breaks down — the few nearest neighbors selected are barely closer than the rest of the dataset, so predictions based on them are little better than using a random subset of points, degrading accuracy for k-NN, clustering, and kernel-based models.

**d) *Suggest dimensionality reduction or regularization techniques to improve performance.***

**Answer:**

- Dimensionality reduction: apply PCA (unsupervised, retains directions of maximum variance) or autoencoders (nonlinear, learned low-dimensional representations) to project the data into a lower-dimensional space before modelling, directly counteracting sparsity.
- Feature selection: use filter methods (mutual information, correlation, ANOVA F-score) to discard uninformative features, or embedded methods such as L1 (Lasso) regularization, which drives irrelevant feature weights exactly to zero while fitting the model.
- Regularization: add L2 (ridge) or L1 penalties (or combine both as Elastic Net) to constrain model complexity and shrink weights, directly limiting the model's ability to fit noise in a sparse, high-dimensional space.
- Cross-validation: use k-fold cross-validation to select the degree of dimensionality reduction / regularization strength based on held-out performance rather than training performance alone, guarding against overfitting during model selection itself.
- Where feasible, collect more training samples or use domain knowledge to hand-engineer a smaller, more informative feature set, directly improving the  $n$ -to- $p$  ratio that underlies the curse of dimensionality.